



JOB SHEET 14

<https://github.com/onlyyourza/PraktikumASD> Tree

14.1 Learning Objectives

After completing this practicum, students will be able to:

1. Understand tree models, particularly binary trees.
2. Design and declare the structure of binary tree algorithms.
3. Apply and implement binary tree algorithms in the context of binary search trees.

14.2 Experiment 1

Implementation of Binary Search Tree Using Linked List (100 Minutes)

14.2.1 Labs Activity

In this experiment, a Binary Search Tree will be implemented using a linked list, focusing on its fundamental operations.

1. In the project created during the previous session, we created a new package named **Week14**.
2. Create the following classes:
 - a. Student00.java
 - b. Node00.java
 - c. BinaryTree00.java
 - d. BinaryTreeMain00.java

Change 00 with your order number.

3. Implement Class Student00, Node00, BinaryTree00 based on the following class diagram:

Student
nim: String name: String className: String ipk: double
Student() Student (nm: String, name: String, kls: String, ipk: double) print(): void

Node
data: Student left: Node right: Node
Node (left: Node, data: Student, right: Node)

BinaryTree
root: Node
BinaryTree() add(data: Student): void



```

find(ipk: double) : boolean
traversePreOrder (node : Node) : void
traversePostOrder (node : Node) void
traverseInOrder (node : Node): void
getSuccessor (del: Node)
delete(ipk: double): void

```

4. In the **Student00** class, declare the attributes according to the **Student** class diagram shown above. Also, add the constructor and methods as specified in the diagram.

```

public class Student00 {
    String nim, name, className;
    double ipk;

    public Student00(){
    }
    public Student00(String nm, String nama, String kls, double ip){
        nim = nm;
        name = nama;
        className = kls;
        ipk = ip;
    }
    void print(){
        System.out.println(nim+" - "+name+" - "+className+" - "+ipk);
    }
}

```

5. In the **Node00** class, add the attributes **data**, **left**, and **right**, along with both a default constructor and a parameterized constructor.

```

public class Node00 {
    Student00 data;
    Node00 left;
    Node00 right;

    Node00(){
    }
    Node00(Student00 data){
        this.data = data;
        left = null;
        right = null;
    }
}

```

6. In the **BinaryTree00** class, add the attribute **root**.

```

public class BinaryTree00 {
    Node00 root;
}

```

7. Add a constructor and the **isEmpty()** method in the **BinaryTree00** class.

```

public BinaryTree00(){
    root = null;
}

```



```

    }

    public boolean isEmpty(){
        return root == null;
    }

```

8. Add the **add()** method in the **BinaryTree00** class. Nodes should be inserted into the binary search tree based on the student's GPA (IPK) value. In this case, the node insertion process is implemented non-recursively to make the insertion flow easier to understand. However, a recursive approach would result in more efficient and concise code.

```

public void add(Student00 data){
    if(isEmpty()){
        root = new Node00(data);
    } else {
        Node00 current = root;
        while(true){
            if(data.ipk < current.data.ipk){
                if(current.left != null){
                    current = current.left;
                } else {
                    current.left = new Node00(data);
                    break;
                }
            } else if(data.ipk > current.data.ipk){
                if(current.right != null){
                    current = current.right;
                } else {
                    current.right = new Node00(data);
                    break;
                }
            } else {
                break;
            }
        }
    }
}

```

9. Add **find()** method

```

public boolean find(double ipk){
    boolean result = false;
    Node00 current = root;
    while(current != null){
        if(current.data.ipk == ipk){
            result = true;
            break;
        } else if(ipk < current.data.ipk){
            current = current.left;
        } else {
            current = current.right;
        }
    }
    return result;
}

```



10. Add the methods **traversePreOrder()**, **traverseInOrder()**, and **traversePostOrder()** to the **BinaryTree00** class. These traversal methods are used to visit and display the nodes in the binary tree using pre-order, in-order, and post-order traversal modes, respectively

```
public void traversePreOrder(Node00 node){
    if(node != null){
        node.data.print();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}
public void traverseInOrder(Node00 node){
    if(node != null){
        traverseInOrder(node.left);
        node.data.print();
        traverseInOrder(node.right);
    }
}
public void traversePostOrder(Node00 node){
    if(node != null){
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.data.print();
    }
}
```

11. Add the method **getSuccessor()**. This method will be used during the deletion process of a node that has two children.

```
Node00 getSuccessor(Node00 del){
    Node00 successor = del.right;
    Node00 successorParent = del;
    while(successor.left != null){
        successorParent = successor;
        successor = successor.left;
    }
    if(successor != del.right){
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}
```

12. Add the **delete()** method. Within this method, include a check to determine whether the tree is empty; if it is not, proceed to locate the node to be deleted.

```
public void delete(double ipk){
    if(isEmpty()){
        System.out.println("Tree is empty!");
        return;
    }
    Node00 parent = root;
    Node00 current = root;
    boolean isLeftChild = false;
    while(current.data.ipk != ipk){
        parent = current;
```



```

        if(ipk < current.data.ipk){
            isLeftChild = true;
            current = current.left;
        } else {
            isLeftChild = false;
            current = current.right;
        }
        if(current == null){
            System.out.println("Couldn't find data!");
            return;
        }
    }
    //delete node with no children
    if(current.left == null && current.right == null){
        if(current == root){
            root = null;
        } else if(isLeftChild){
            parent.left = null;
        } else {
            parent.right = null;
        }
    } else if(current.right == null){//delete node with a left child
        if(current == root){
            root = current.left;
        } else if(isLeftChild){
            parent.left = current.left;
        } else {
            parent.right = current.left;
        }
    } else if(current.left == null){//delete node with a right child
        if(current == root){
            root = current.right;
        } else if(isLeftChild){
            parent.left = current.right;
        } else {
            parent.right = current.right;
        }
    } else //delete node with 2 children
        Node00 successor = getSuccessor(current);
        if(current == root){
            root = successor;
        } else if(isLeftChild){
            parent.left = successor;
        } else {
            parent.right = successor;
        }
        successor.left = current.left;
    }
}

```

13. Open the **BinaryTreeMain00** class and add the **main()** method. Then, include the following code:

```

public class BinaryTreeMain00 {
    public static void main(String[] args) {
        BinaryTree00 bst = new BinaryTree00();
        bst.add(new Student00("244107020138", "Devin", "TI-1I", 3.57));
        bst.add(new Student00("244107020023", "Dewi", "TI-1I", 3.85));
    }
}

```



```

bst.add(new Student00("244107020225", "Wahyu", "TI-1I", 3.21));
bst.add(new Student00("244107020076", "Angelina", "TI-1I", 3.54));

System.out.println("Student list (in-order traversal)");
bst.traverseInOrder(bst.root);

System.out.println("Search data");
System.out.print("Search a student with IPK: 3.54: ");
String result = bst.find(3.54) ? "Found" : "Not Found";
System.out.println(result);

System.out.print("Search a student with IPK: 3.22: ");
result = bst.find(3.22) ? "Found" : "Not Found";
System.out.println(result);

bst.add(new Student00("244107020223", "Andhika", "TI-1I", 3.72));
bst.add(new Student00("244107020226", "Bima", "TI-1I", 3.37));
bst.add(new Student00("244107020181", "Eiyu", "TI-1I", 3.46));
System.out.println("Student list:");
System.out.println("In-order traversal:");
bst.traverseInOrder(bst.root);
System.out.println("Pre-order traversal:");
bst.traversePreOrder(bst.root);
System.out.println("Post-order traversal:");
bst.traversePostOrder(bst.root);

System.out.println("Data deletion");
bst.delete(3.57);
System.out.println("Student list after deletion:");
bst.traverseInOrder(bst.root);
}
}

```

14. Compile and run the **BinaryTreeMain00** class to simulate the execution of the implemented binary tree program.
15. Observe and analyze the output generated during the program run.

```

Student list (in-order traversal)
244107020225 - Wahyu - TI-1I - 3.21
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020023 - Dewi - TI-1I - 3.85
Search data
Search a student with IPK: 3.54: Found
Search a student with IPK: 3.22: Not Found
Student list:
In-order traversal:
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
Pre-order traversal:
244107020138 - Devin - TI-1I - 3.57
244107020225 - Wahyu - TI-1I - 3.21
244107020076 - Angelina - TI-1I - 3.54
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020023 - Dewi - TI-1I - 3.85
244107020223 - Andhika - TI-1I - 3.72

```



```

Post-order traversal:
244107020181 - Eiyu - TI-1I - 3.46
244107020226 - Bima - TI-1I - 3.37
244107020076 - Angelina - TI-1I - 3.54
244107020225 - Wahyu - TI-1I - 3.21
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
244107020138 - Devin - TI-1I - 3.57
Data deletion
Student list after deletion:
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020076 - Angelina - TI-1I - 3.54
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85

```

```

Jobsheet14 > Student14.java > Student14
1  package Jobsheet14;
2
3  public class Student14 {
4      String nim, name, className;
5      double ipk;
6
7      public Student14() {
8      }
9
10     public Student14(String nm, String nama, String kls, double ip) {
11         nim = nm;
12         name = nama;
13         className = kls;
14         ipk = ip;
15     }
16
17     void print() {
18         System.out.println(nim + " - " + name + " - " + className + " - " + ipk);
19     }
20 }

```

```

Jobsheet14 > Node14.java > Node14
1  package Jobsheet14;
2
3  public class Node14 {
4      Student14 data;
5      Node14 left;
6      Node14 right;
7
8      Node14() {
9      }
10
11     Node14(Student14 data) {
12         this.data = data;
13         left = null;
14         right = null;
15     }
16 }

```



```

1  package Jobsheet14;
2
3  public class BinaryTree14 {
4      Node14 root;
5
6      public BinaryTree14() {
7          root = null;
8      }
9
10     public boolean isEmpty() {
11         return root == null;
12     }
13
14     public void add(Student14 data) {
15         if (isEmpty()) {
16             root = new Node14(data);
17         } else {
18             Node14 current = root;
19             while (true) {
20                 if (data.ipk < current.data.ipk) {
21                     if (current.left != null) {
22                         current = current.left;
23                     } else {
24                         current.left = new Node14(data);
25                         break;
26                     }
27                 } else if (data.ipk > current.data.ipk) {
28                     if (current.right != null) {
29                         current = current.right;
30                     } else {
31                         current.right = new Node14(data);
32                         break;
33                     }
34                 } else {
35                     break;
36                 }
37             }
38         }
39     }
40 }

```

```

41 // Assignment 1: recursive add
42 public void addRekursif(Student14 data) {
43     root = addRekursif(root, data);
44 }
45
46 private Node14 addRekursif(Node14 current, Student14 data) {
47     if (current == null) {
48         return new Node14(data);
49     }
50     if (data.ipk < current.data.ipk) {
51         current.left = addRekursif(current.left, data);
52     } else if (data.ipk > current.data.ipk) {
53         current.right = addRekursif(current.right, data);
54     }
55     return current;
56 }
57
58 // Assignment 2: Lowest IPK (Leftmost node)
59 public void getMinIPK() {
60     if (isEmpty()) {
61         System.out.println("Tree is empty!");
62         return;
63     }
64     Node14 current = root;
65     while (current.left != null) {
66         current = current.left;
67     }
68     System.out.print("Student with lowest IPK: ");
69     current.data.print();
70 }
71

```



```

72 // Assignment 2: highest IPK (rightmost node)
73 public void getMaxIPK() {
74     if (isEmpty()) {
75         System.out.println(x: "Tree is empty!");
76         return;
77     }
78     Node14 current = root;
79     while (current.right != null) {
80         current = current.right;
81     }
82     System.out.print(s: "Student with highest IPK: ");
83     current.data.print();
84 }
85
86 // Assignment 3: display students with IPK above a threshold
87 public void displayStudentsWithIPKAbove(double threshold) {
88     System.out.println("Students with IPK above " + threshold + ":");
89     displayStudentsWithIPKAbove(root, threshold);
90 }
91
92 private void displayStudentsWithIPKAbove(Node14 node, double threshold) {
93     if (node != null) {
94         displayStudentsWithIPKAbove(node.left, threshold);
95         if (node.data.ipk > threshold) {
96             node.data.print();
97         }
98         displayStudentsWithIPKAbove(node.right, threshold);
99     }
100 }
101
102 public boolean find(double ipk) {
103     boolean result = false;
104     Node14 current = root;
105     while (current != null) {
106         if (current.data.ipk == ipk) {
107             result = true;
108             break;
109         }
110         if (ipk < current.data.ipk) {
111             current = current.left;
112         } else {
113             current = current.right;
114         }
115     }
116     return result;
117 }
118
119 public void traversePostOrder(Node14 node) {
120     if (node != null) {
121         traversePostOrder(node.left);
122         traversePostOrder(node.right);
123         node.data.print();
124     }
125 }
126
127 Node14 getSuccessor(Node14 del) {
128     Node14 successor = del.right;
129     Node14 successorParent = del;
130     while (successor.left != null) {
131         successorParent = successor;
132         successor = successor.left;
133     }
134     if (successor != del.right) {
135         successorParent.left = successor.right;
136         successor.right = del.right;
137     }
138     return successor;
139 }
140
141 public void delete(double ipk) {
142     if (isEmpty()) {
143         System.out.println(x: "Tree is empty!");
144         return;
145     }
146     Node14 parent = root;
147     Node14 current = root;
148     boolean isLeftChild = false;
149     while (current.data.ipk != ipk) {
150         parent = current;
151         if (ipk < current.data.ipk) {
152             isLeftChild = true;
153             current = current.left;
154         } else {
155             current = current.right;
156         }
157     }
158     if (current == root) {
159         root = getSuccessor(current);
160     } else {
161         if (isLeftChild) {
162             parent.left = getSuccessor(current);
163         } else {
164             parent.right = getSuccessor(current);
165         }
166     }
167 }

```



```

102     public boolean find(double ipk) {
103         boolean result = false;
104         Node14 current = root;
105         while (current != null) {
106             if (current.data.ipk == ipk) {
107                 result = true;
108                 break;
109             } else if (ipk < current.data.ipk) {
110                 current = current.left;
111             } else {
112                 current = current.right;
113             }
114         }
115         return result;
116     }
117
118     public void traversePreOrder(Node14 node) {
119         if (node != null) {
120             node.data.print();
121             traversePreOrder(node.left);
122             traversePreOrder(node.right);
123         }
124     }
125
126     public void traverseInOrder(Node14 node) {
127         if (node != null) {
128             traverseInOrder(node.left);
129             node.data.print();
130             traverseInOrder(node.right);
131         }
132     }
133
134     public void traversePostOrder(Node14 node) {
135         if (node != null) {
136             traversePostOrder(node.left);
137             traversePostOrder(node.right);
138
139             node.data.print();
140         }
141     }
142
143     Node14 getSuccessor(Node14 del) {
144         Node14 successor = del.right;
145         Node14 successorParent = del;
146         while (successor.left != null) {
147             successorParent = successor;
148             successor = successor.left;
149         }
150         if (successor != del.right) {
151             successorParent.left = successor.right;
152             successor.right = del.right;
153         }
154         return successor;
155     }
156
157     public void delete(double ipk) {
158         if (isEmpty()) {
159             System.out.println(x: "Tree is empty!");
160             return;
161         }
162         Node14 parent = root;
163         Node14 current = root;
164         boolean isLeftChild = false;
165         while (current.data.ipk != ipk) {
166             parent = current;
167             if (ipk < current.data.ipk) {
168                 isLeftChild = true;
169                 current = current.left;
170             } else {
171                 isLeftChild = false;
172                 current = current.right;
173             }
174         }
175         if (current == root) {
176             root = getSuccessor(current);
177         } else {
178             if (isLeftChild) {
179                 parent.left = getSuccessor(current);
180             } else {
181                 parent.right = getSuccessor(current);
182             }
183         }
184     }

```



```

178 // delete node with no children
179 if (current.left == null && current.right == null) {
180     if (current == root) {
181         root = null;
182     } else if (isLeftChild) {
183         parent.left = null;
184     } else {
185         parent.right = null;
186     }
187 } else if (current.right == null) { // delete node with a left child
188     if (current == root) {
189         root = current.left;
190     } else if (isLeftChild) {
191         parent.left = current.left;
192     } else {
193         parent.right = current.left;
194     }
195 } else if (current.left == null) { // delete node with a right child
196     if (current == root) {
197         root = current.right;
198     } else if (isLeftChild) {
199         parent.left = current.right;
200     } else {
201         parent.right = current.right;
202     }
203 } else { // delete node with 2 children
204     Node14 successor = getSuccessor(current);
205     if (current == root) {
206         root = successor;
207     } else if (isLeftChild) {
208         parent.left = successor;
209     } else {
210         parent.right = successor;
211 }
}

3 public class BinaryTreeMain14 {
4     public static void main(String[] args) {
5         BinaryTree14 bst = new BinaryTree14();
6         bst.add(new Student14(nm: "244107020138", nama: "Devin", kls: "TI-11", ip: 3.57));
7         bst.add(new Student14(nm: "244107020023", nama: "Dewi", kls: "TI-11", ip: 3.85));
8         bst.add(new Student14(nm: "244107020225", nama: "Wahyu", kls: "TI-11", ip: 3.21));
9         bst.add(new Student14(nm: "244107020076", nama: "Angelina", kls: "TI-11", ip: 3.54));
10
11         System.out.println("Student list (in-order traversal);");
12         bst.traverseInOrder(bst.root);
13
14         System.out.println("Search data");
15         System.out.print("Search a student with IPK: 3.54: ");
16         String result = bst.find(ipk: 3.54) ? "Found" : "Not Found";
17         System.out.println(result);
18
19         System.out.print("Search a student with IPK: 3.22: ");
20         result = bst.find(ipk: 3.22) ? "Found" : "Not Found";
21         System.out.println(result);
22
23         bst.add(new Student14(nm: "244107020223", nama: "Andhika", kls: "TI-11", ip: 3.72));
24         bst.add(new Student14(nm: "244107020226", nama: "Bima", kls: "TI-11", ip: 3.37));
25         bst.add(new Student14(nm: "244107020181", nama: "Eiyu", kls: "TI-11", ip: 3.46));
26         System.out.println("Student list:");
27         System.out.println("In-order traversal:");
28         bst.traverseInOrder(bst.root);
29         System.out.println("Pre-order traversal:");
30         bst.traversePreOrder(bst.root);
31         System.out.println("Post-order traversal:");
32         bst.traversePostOrder(bst.root);
33
34         System.out.println("Data deletion");
35         bst.delete(ipk: 3.57);
36         System.out.println("Student list after deletion:");
37         bst.traverseInOrder(bst.root);
38     }
39 }

```



```
Student list (in-order traversal)
244107020225 - Wahyu - TI-1I - 3.21
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020023 - Dewi - TI-1I - 3.85
Search data
Search a student with IPK: 3.54: Found
Search a student with IPK: 3.22: Not Found
Student list:
In-order traversal:
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
Pre-order traversal:
244107020138 - Devin - TI-1I - 3.57
244107020225 - Wahyu - TI-1I - 3.21
244107020076 - Angelina - TI-1I - 3.54
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020023 - Dewi - TI-1I - 3.85
244107020223 - Andhika - TI-1I - 3.72
Post-order traversal:
244107020181 - Eiyu - TI-1I - 3.46
244107020226 - Bima - TI-1I - 3.37
244107020076 - Angelina - TI-1I - 3.54
244107020225 - Wahyu - TI-1I - 3.21
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
244107020138 - Devin - TI-1I - 3.57
Data deletion
Student list after deletion:
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
```



14.2.2 Questions

1. Why is data search in a binary search tree more efficient compared to a regular binary tree?
2. What are the purposes of the **left** and **right** attributes in the **Node** class?
3. a. What is the function of the **root** attribute in the **BinaryTree** class?
b. When a **BinaryTree** object is first created, what is the initial value of **root**?
4. When the tree is empty and a new node is to be added, what process takes place?
5. Consider the following line of code inside the **add()** method. Explain in detail the purpose of this line of code.

```
if(data.ipk < current.data.ipk){  
    if(current.left != null){  
        current = current.left;  
    } else {  
        current.left = new Node00(data);  
        break;  
    }  
} else if(data.ipk > current.data.ipk){  
    if(current.right != null){  
        current = current.right;  
    } else {  
        current.right = new Node00(data);  
        break;  
    }  
}
```

6. Explain the steps involved in the **delete()** method when removing a node that has two children. How does the **getSuccessor()** method assist in this process?



Answer

1. Why is data search in a binary search tree more efficient compared to a regular binary tree? In a BST, nodes are ordered: every left subtree holds smaller values and every right subtree holds larger values. This lets the search discard half of the remaining tree at each step, giving $O(\log n)$ on a balanced tree. A regular binary tree has no ordering, so a search must potentially visit every node, costing $O(n)$.
2. What are the purposes of the left and right attributes in the Node class? They are references (pointers) to the node's child nodes: left points to the subtree containing smaller IPK values, and right points to the subtree containing larger IPK values. They link the nodes together to form the tree structure.
3. A. What is the function of the root attribute in the BinaryTree class? It is the reference to the topmost node of the tree, serving as the single entry point from which all other nodes are reached during insertion, search, traversal, and deletion.
B. When a BinaryTree object is first created, what is the initial value of root? null, because the constructor sets root = null, indicating the tree is empty.
4. When the tree is empty and a new node is to be added, what process takes place? isEmpty() returns true, so the add() method creates a new node from the data and assigns it directly to root. This first node becomes the root of the tree.
5. Explain in detail the purpose of the highlighted code inside add(): This block decides where the new data belongs relative to the current node. If data.ipk is smaller than the current node's IPK, it should go to the left: if a left child already exists, current moves down to it to keep searching; if not, a new node is created as the left child and the loop stops. The mirror logic applies when data.ipk is greater: it moves right, and if there is no right child, the new node is attached as the right child. This preserves the BST ordering property (smaller left, larger right).
6. Explain the steps in delete() when removing a node with two children. How does getSuccessor() help? When the target node has two children, it cannot simply be removed. The method finds the in-order successor, the smallest value in the right subtree via getSuccessor(). getSuccessor() starts at del.right and walks left as far as possible to reach that smallest node; if the successor is not the direct right child, it relinks the successor's parent and reattaches the deleted node's right subtree to the successor. The successor then replaces the deleted node (becoming root or the parent's child as appropriate), and the deleted node's left subtree is attached to the successor's left. This keeps the BST ordering intact.

14.3 Experiment 2

Implementation of Binary Tree Using Array (45 Minutes)

14.3.1 Labs Activity

1. In this binary tree implementation experiment using an array, the tree data is stored directly in an array and initialized within the main() method. Subsequently, the in-order traversal process will be simulated.
2. Create the classes BinaryTreeArray00 and BinaryTreeArrayMain00, replacing 00 with your attendance number.
3. In the BinaryTreeArray00 class, create the attributes data and idxLast. Also, implement the methods populateData() and traverseInOrder().



```
public class BinaryTreeArray00 {
    Student00[] data;
    int idxLast;
    public BinaryTreeArray00(){
        data = new Student00[10];
        idxLast = -1;
    }
    void populateData(Student00[] data, int idxLast){
        this.data = data;
        this.idxLast = idxLast;
    }
    void traverseInOrder(int idxStart){
        if(idxStart <= idxLast){
            if(data[idxStart] != null){
                traverseInOrder(2 * idxStart + 1);
                data[idxStart].print();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }
}
```

4. Then, in the **BinaryTreeArrayMain00** class, create the **main()** method and insert the code shown in the following image inside the method.

```
public class BinaryTreeArrayMain00 {
    public static void main(String[] args) {
        BinaryTreeArray00 bta = new BinaryTreeArray00();
        Student00 m1 = new Student00("244107020138", "Devin", "TI-1I", 3.57);
        Student00 m2 = new Student00("244107020023", "Dewi", "TI-1I", 3.85);
        Student00 m3 = new Student00("244107020225", "Wahyu", "TI-1I", 3.21);
        Student00 m4 = new Student00("244107020076", "Angelina", "TI-1I", 3.54);
        Student00 m5 = new Student00("244107020223", "Andhika", "TI-1I", 3.72);
        Student00 m6 = new Student00("244107020226", "Bima", "TI-1I", 3.37);
        Student00 m7 = new Student00("244107020181", "Eiyu", "TI-1I", 3.46);

        Student00[] data = {m1, m2, m3, m4, m5, m6, m7};
        bta.populateData(data, data.length-1);
        System.out.println("In-order traversal:");
        bta.traverseInOrder(0);
    }
}
```

5. Run the **BinaryTreeArrayMain00** class and observe the output!

```
In-order traversal:
244107020076 - Angelina - TI-1I - 3.54
244107020023 - Dewi - TI-1I - 3.85
244107020223 - Andhika - TI-1I - 3.72
244107020138 - Devin - TI-1I - 3.57
244107020226 - Bima - TI-1I - 3.37
244107020225 - Wahyu - TI-1I - 3.21
244107020181 - Eiyu - TI-1I - 3.46
```



```

1  package Jobsheet14;
2
3  public class BinaryTreeArray14 {
4      Student14[] data;
5      int idxLast;
6
7      public BinaryTreeArray14() {
8          data = new Student14[10];
9          idxLast = -1;
10     }
11
12     void populateData(Student14[] data, int idxLast) {
13         this.data = data;
14         this.idxLast = idxLast;
15     }
16
17     // Assignment 4: insert data into array-based BST
18     void add(Student14 data) {
19         idxLast++;
20         if (idxLast >= this.data.length) {
21             System.out.println(x: "Array is full!");
22             idxLast--;
23             return;
24         }
25         this.data[idxLast] = data;
26     }
27
28     // Assignment 4: pre-order traversal
29     void traversePreOrder(int idxStart) {
30         if (idxStart <= idxLast) {
31             if (data[idxStart] != null) {
32                 data[idxStart].print();
33                 traversePreOrder(2 * idxStart + 1);
34                 traversePreOrder(2 * idxStart + 2);
35             }
36         }
37     }

```

```

39     void traverseInOrder(int idxStart) {
40         if (idxStart <= idxLast) {
41             if (data[idxStart] != null) {
42                 traverseInOrder(2 * idxStart + 1);
43                 data[idxStart].print();
44                 traverseInOrder(2 * idxStart + 2);
45             }
46         }
47     }
48 }
49

```



```
1 package Jobsheet14;
2
3 public class BinaryTreeArrayMain14 {
4     Run | Debug
5     BinaryTreeArray14 bta = new BinaryTreeArray14();
6     Student14 m1 = new Student14(nm: "244107020138", nama: "Devin", kls: "TI-1I", ip: 3.57);
7     Student14 m2 = new Student14(nm: "244107020023", nama: "Dewi", kls: "TI-1I", ip: 3.85);
8     Student14 m3 = new Student14(nm: "244107020225", nama: "Wahyu", kls: "TI-1I", ip: 3.21);
9     Student14 m4 = new Student14(nm: "244107020076", nama: "Angelina", kls: "TI-1I", ip: 3.54);
10    Student14 m5 = new Student14(nm: "244107020223", nama: "Andhika", kls: "TI-1I", ip: 3.72);
11    Student14 m6 = new Student14(nm: "244107020226", nama: "Bima", kls: "TI-1I", ip: 3.37);
12    Student14 m7 = new Student14(nm: "244107020181", nama: "Eiyu", kls: "TI-1I", ip: 3.46);
13
14    Student14[] data = {m1, m2, m3, m4, m5, m6, m7};
15    bta.populateData(data, data.length - 1);
16    System.out.println(x: "In-order traversal:");
17    bta.traverseInOrder(idxStart: 0);
18 }
19 }
20
```



14.3.2 Questions

1. What is the purpose of the **data** and **idxLast** attributes in the **BinaryTreeArray** class?
2. What is the function of the **populateData()** method?
3. What is the purpose of the **traverseInOrder()** method?
4. If a binary tree node is stored at index **2** in the array, at which indices are its left and right children located, respectively?
5. What is the purpose of the statement `int idxLast = 6` in Experiment 2, step 4?
6. Why are the indices **$2 * \text{idxStart} + 1$** and **$2 * \text{idxStart} + 2$** used in the recursive calls, and how do they relate to the structure of a binary tree represented as an array?

Answer

1. What is the purpose of the **data** and **idxLast** attributes in the **BinaryTreeArray** class? **data** is the array that stores the tree nodes, where the array index encodes each node's position in the tree. **idxLast** records the index of the last filled element, marking the valid boundary of the data so traversal does not read empty/out-of-range slots.
2. What is the function of the **populateData()** method? It fills the tree by assigning an externally prepared array of students and its last valid index into the object's **data** and **idxLast** fields, initializing the array-based tree in one step.
3. What is the purpose of the **traverseInOrder()** method? It visits the nodes in in-order sequence (left child, node, right child) recursively using index arithmetic, printing each node so the data is output in left-root-right order.
4. If a node is stored at index **2**, at which indices are its left and right children located? Left child at index $2 * 2 + 1 = 5$, right child at index $2 * 2 + 2 = 6$.
5. What is the purpose of the statement `int idxLast = 6` (i.e. `data.length-1`) in step 4? It marks the index of the last valid element in the array (7 elements, indices 0–6). It tells the traversal where the real data ends so recursion stops at the correct boundary instead of processing empty slots.
6. Why are $2 * \text{idxStart} + 1$ and $2 * \text{idxStart} + 2$ used in the recursive calls, and how do they relate to the array representation of a binary tree? In an array-based binary tree, a node at index *i* has its left child at $2 * i + 1$ and right child at $2 * i + 2$ (and parent at $(i-1)/2$). These formulas come from the level-order numbering of a complete binary tree, letting the structure be navigated purely by index without explicit pointers. The two recursive calls therefore move to the current node's left and right children.



14.4 Assignments

Times Allocated: 150 minutes

1. Implement a recursive method **addRekursif()** in the **BinaryTree00** class to add nodes recursively.

```

41 // Assignment 1: recursive add
42 public void addRekursif(Student14 data) {
43     root = addRekursif(root, data);
44 }
45
46 private Node14 addRekursif(Node14 current, Student14 data) {
47     if (current == null) {
48         return new Node14(data);
49     }
50     if (data.ipk < current.data.ipk) {
51         current.left = addRekursif(current.left, data);
52     } else if (data.ipk > current.data.ipk) {
53         current.right = addRekursif(current.right, data);
54     }
55     return current;
56 }
57

```

2. Create methods **getMiniIPK()** and **getMaxIPK()** in the **BinaryTree00** class to retrieve and display the student data with the lowest and highest GPA(IPK) values stored in the binary search tree.

```

58 // Assignment 2: lowest IPK (leftmost node)
59 public void getMinIPK() {
60     if (isEmpty()) {
61         System.out.println(x: "Tree is empty!");
62         return;
63     }
64     Node14 current = root;
65     while (current.left != null) {
66         current = current.left;
67     }
68     System.out.print(s: "Student with lowest IPK: ");
69     current.data.print();
70 }
71
72 // Assignment 2: highest IPK (rightmost node)
73 public void getMaxIPK() {
74     if (isEmpty()) {
75         System.out.println(x: "Tree is empty!");
76         return;
77     }
78     Node14 current = root;
79     while (current.right != null) {
80         current = current.right;
81     }
82     System.out.print(s: "Student with highest IPK: ");
83     current.data.print();
84 }
85

```

3. Develop a method **displayStudentsWithIPKAbove (double threshold)** in the **BinaryTree00** class to display student data whose GPA(IPK) exceeds a specified threshold (e.g., above 3.50) within the binary search tree.



```

86 // Assignment 3: display students with IPK above a threshold
87 public void displayStudentsWithIPKAbove(double threshold) {
88     System.out.println("Students with IPK above " + threshold + ":");
89     displayStudentsWithIPKAbove(root, threshold);
90 }
91
92 private void displayStudentsWithIPKAbove(Node14 node, double threshold) {
93     if (node != null) {
94         displayStudentsWithIPKAbove(node.left, threshold);
95         if (node.data.ipk > threshold) {
96             node.data.print();
97         }
98         displayStudentsWithIPKAbove(node.right, threshold);
99     }
100 }

```

4. Modify the **BinaryTreeArray00** class by adding the following methods:
- **add(Student data)** to insert data into the binary tree represented as an array.
 - **traversePreOrder()** to perform a pre-order traversal of the binary tree.

```

17 // Assignment 4: insert data into array-based BST
18 void add(Student14 data) {
19     idxLast++;
20     if (idxLast >= this.data.length) {
21         System.out.println(x: "Array is full!");
22         idxLast--;
23         return;
24     }
25     this.data[idxLast] = data;
26 }
27
28 // Assignment 4: pre-order traversal
29 void traversePreOrder(int idxStart) {
30     if (idxStart <= idxLast) {
31         if (data[idxStart] != null) {
32             data[idxStart].print();
33             traversePreOrder(2 * idxStart + 1);
34             traversePreOrder(2 * idxStart + 2);
35         }
36     }
37 }

```